

PROJECT GUIDE

Simulating and Optimizing Emergency-Department Patient Flow

CSE 402 — Numerical Analysis, Simulation and Modeling Sessional ·
Section C

Sudip Kumar Saha	2105152	WP1 — baseline DES engine & validation
Ahnaf Akib Tanim	2105154	WP2 — c·μ / WSEPT scheduling rule
Md. Shahjalal Rumman	2105165	WP3 — Nelder-Mead optimiser
Md. Dulal Hossain	2105169	WP4 — cost-based staffing
Md. Shoaib Hossain	2105170	WP5 — common random numbers & final analysis

What this document is

Everything you need to understand, run, explain and defend this project, in one place: what the base paper does, what we added, how each method works, which file does what, how to run it, and what the results mean. Every number in here is read straight out of the generated results/ folder, so the guide cannot drift from the code.

The project in five numbers

6/6 structural findings of the paper reproduced	19× fewer simulations than the paper's grid search	99.0% variance removed by common random numbers	28.9k weekly cost saved by the optimal roster	33 automated correctness checks, all passing
---	--	---	---	--

Base paper. Lv, W.; Liu, R.; Yan, F.; Wang, Y. (2026). *Discrete Event Simulation-Based Analysis and Optimization of Emergency Patient Scheduling Strategies*. Healthcare 14(1), 99. doi:10.3390/healthcare14010099

Generated 2026-09-24 by docs/make_guide.py from the results of python scripts/run_all.py (seed 20250402, 100 replications per policy).

What is in this guide

§	Section	Covers
1	The 60-second summary	What we did and what we found
2	The base paper	The system, the three policies, the conclusions, the gaps
3	What we built	The five work packages
4	How the simulator works	Patient journey, event loop, module map
5	The methods, explained	Every numerical technique, in plain language
6	Project map	Which folder and file does what
7	How to run everything	Commands, flags, runtimes
8	Results and analysis	Work package by work package, with figures
9	Paper vs. ours	Side-by-side, where we agree and where we differ
10	Judgement calls	The five things the paper left unspecified
11	Viva preparation	Likely questions with short answers
12	Glossary and references	Every symbol and term used

How to read this if you are in a hurry
Sections 1, 8 and 9 are the results. Sections 2–5 are the method. Sections 6–7 are the code and how to run it. Section 11 is exam prep. If you read only one page, read section 1.

1. The 60-second summary

A hospital emergency department has a fixed number of doctors and three queues of patients competing for them: urgent (Level III) new arrivals, non-urgent (Level IV) new arrivals, and patients coming back for a follow-up after an X-ray or blood test. **Which queue should the next free doctor take from?** That single decision is the whole problem.

The base paper builds a computer simulation of such a department and compares three answers to that question. We rebuilt that simulation from scratch, checked it against every number the paper published, and then extended it in four directions the paper explicitly does not attempt.

What we did

#	Work package	In one sentence	Headline result
WP1	Rebuild & validate	Re-implement the paper's model from scratch and check it against every published KPI.	All 6 structural findings reproduced; equipment table within 0.5 pp.
WP2	A better-founded policy	Add the c-μ rule, which is <i>provably</i> optimal, instead of another hand-made heuristic.	Lowest holding cost and lowest mean wait of all five policies.
WP3	A real optimiser	Replace the paper's brute-force grid search with a hand-written Nelder-Mead simplex.	Same answer for 19× fewer simulations (50,415 → 2,650).
WP4	How many doctors?	Put a price on waiting and on salary, then search over staffing rosters.	Roster (6, 6, 3) beats the paper's (5,5,3) by 20.7% of weekly cost.
WP5	Sharper statistics	Give every policy the identical simulated week (common random numbers).	99.0% of comparison variance removed; 96× more efficient.

What we found

The rebuild is faithful in structure, not in absolute level. Our waiting times run 12–25% above the paper's. That is expected and explainable: the paper's published arrival table, service-time distributions and staffing levels together imply a system load of $\rho \approx 0.87$, and at that load the mean wait is extremely sensitive to the last percentage point of utilisation. The source dataset is not public, so an exact numeric match is not attainable and we do not claim one. Everything *structural* does reproduce: the policy ranking, the per-level trade-offs, near-identical doctor utilisation across policies, the equipment-utilisation table, and every sensitivity-analysis finding.

The single most interesting finding

On our rebuilt model, **the paper's own tuned thresholds $(k_1, k_2) = (13.1, 2.1)$ break the Level IV service-level constraint** — they reach only 89.4% against a 95% target. The reason is mechanical: $k_2 = 2.1$ means a Level IV patient is only flagged urgent after waiting 117.9 of their 120 allowed minutes, which is too late to rescue them on a busier system. Re-tuning under the constraint (WP3) moves the optimum to a much larger k_2 and produces the only fast policy a hospital could actually adopt. **This is the argument for re-tuning rather than copying published parameters.**

The final answer depends on what you are optimising

There is no single winner, and pretending otherwise would be the easiest way to get this wrong. Three honest answers:

If you care about...	The best policy is	Because
Raw average waiting time	WSEPT (42.32 min)	it serves the class with the best cost-times-speed index first
Waiting measured against each patient's clinical tolerance	WSEPT (0.3763)	this is exactly the objective the c·μ rule is proven optimal for
Meeting the service-level target (what a hospital must do)	SBP* (44.18 min)	only IFP and SBP* clear 95% on <i>both</i> triage levels

SBP* is our re-tuned Slack-Based Policy from WP3. It is 16% faster than the paper's baseline IFP *and* it satisfies the constraint — which the paper's own published parameters do not, on our system.

2. The base paper: what it actually does

2.1 The system being modelled

A Grade-A tertiary hospital emergency department in China. Only triage **Level III** (urgent) and **Level IV** (non-urgent) patients are modelled — Levels I and II are critical and go straight to resuscitation, so they never queue. Each patient follows the same path:

1	Arrive	Arrivals follow a rate that changes every hour and repeats weekly (busy 09:00–11:00 and 19:00–21:00, dead at 04:00). 25% are Level III, 75% Level IV.
2	Wait for a doctor	Join the Level III or Level IV <i>initial</i> queue. This wait is what the triage target governs: 30 min for Level III, 120 min for Level IV.
3	Initial consultation	A doctor sees them for 5–15 min (truncated exponential, mean ≈ 9.09 min).
4	Diagnostics (60% of patients)	Lab, X-ray, CT and/or ultrasound. Each machine can serve one patient at a time, and each result takes time to come back (20 min for lab, 30 for X-ray and CT, instant for ultrasound).
5	Wait again	Once <i>all</i> results are in, the patient joins the follow-up queue — competing with new arrivals for the same doctors.
6	Follow-up consultation	5–25 min (mean ≈ 12.84 min), then discharge.

Doctors work three shifts: **5** from 07:00–15:00, **5** from 15:00–22:00, **3** from 22:00–07:00. A doctor never abandons a patient at shift change. One simulated run covers one full week (10,080 minutes, about 2,280 patients).

2.2 The three policies it compares

All three answer the same question — a doctor is free, which queue do they take from? — and differ only in the answer.

Policy	The rule, in one line	What it does well	What it breaks
IFP Initial-First	Always serve new arrivals first: Level III, then Level IV, then follow-ups.	Nobody ever breaches their triage target.	Follow-up patients are starved; they wait ~ 98 min, which drags the overall average up.
ALT Alternating 1:1	At most half the doctors may be on new arrivals at once; the rest take follow-ups.	Level III waits collapse to ~ 6 min; follow-ups clear fast.	Level IV new arrivals are squeezed and 13% of them breach the 120-min target.
SBP Slack-Based	A patient becomes 'urgent' once they have waited $T_l - k_l$ minutes. Priority: urgent L3 > urgent L4 > follow-ups > everyone else.	Two dials (k_1, k_2) let you trade the two levels against each other; the paper tunes them.	Its performance depends entirely on the tuning — which is exactly what WP3 exploits.

The slack idea in one sentence

kl is a *head start*: with $T_3 = 30$ min and $k_1 = 13.1$, a Level III patient is promoted to urgent after $30 - 13.1 = 16.9$ minutes of waiting, so the system starts rescuing them 13.1 minutes before they would actually breach. Bigger k = earlier rescue = more protection for that class, at the cost of everyone else.

2.3 What the paper concludes

Claim	Its evidence
SBP is best, cutting mean wait 46.26 → 35.25 min (−23.8%)	100 replications per policy, paired t-tests, Cohen's d > 1.5
Tuned thresholds are $(k_1, k_2) = (13.1, 2.1)$	Two-stage brute-force grid search: coarse step 1.0, then fine step 0.1
Doctors, not machines, are the bottleneck	Equipment utilisation 14–19% vs. doctor utilisation ~85%
Results are robust	Arrival rate $\pm 15\%$, seven staffing rosters, lognormal service times

2.4 The four gaps we fill

The paper does	The paper does not do	Our answer
Grid-search two thresholds	Optimise the staffing level at the same time	WP4 — a cost model plus a roster search
Compare three heuristics	Compare against a provably optimal queueing rule	WP2 — the c·μ rule and its generalisation
Brute force (hundreds of full runs)	Use a derivative-free numerical optimiser	WP3 — hand-written Nelder-Mead + penalty function
100 independent replications	Apply variance reduction	WP5 — exact common random numbers
Report results	Release the engine or the data	WP1 — an open, validated, from-scratch rebuild

3. What we built

About 7,850 lines of Python: a simulation library, nine runnable experiments, and a 33-check test suite. The simulation engine uses a hand-rolled priority-queue event loop — **no SimPy or other black-box process library** — so every mechanism described in the paper is visible and checkable in the code.

Work package	Owner	What it adds	Script that produces it
WP1 Baseline engine & validation	Sudip	The whole simulator, plus a check against every published KPI, t-test and equipment number.	run_01_validate_paper.py
WP2 c:μ / WSEPT rule	Ahnaf	Two new policies derived from queueing theory rather than invented, and a holding-cost metric to judge them fairly.	run_03_wsept_policy.py
WP3 Nelder-Mead optimiser	Rumman	A hand-written simplex method with a penalty function, benchmarked against the paper's grid search on equal terms.	run_02_grid_search.py run_04_nelder_mead.py
WP4 Cost-optimal staffing	Dulal	A weekly cost function and a search over integer rosters with the thresholds re-tuned inside each one.	run_05_staffing_cost.py
WP5 CRN & final analysis	Shoaib	Exact common random numbers, the variance reduction quantified, and the consolidated comparison.	run_06_crn_variance.py run_08_final_comparison.py

Two supporting scripts

run_00_demo.py	Shows the engine working: a live event-by-event trace, then a console dashboard of queue lengths updating every simulated hour, then all five policies on one shared week. Run this first.
run_07_sensitivity.py	Reproduces the paper's robustness study (arrival rate $\pm 15\%$, the seven staffing rosters, lognormal service times), extended to our two new policies.

4. How the simulator works

4.1 Discrete-event simulation in one idea

Nothing interesting happens between events. A patient arriving, a consultation ending, a scan finishing — those are the only moments the state of the department changes. So instead of ticking a clock forward minute by minute, we keep a **future-event list**: a min-heap of things that are going to happen, sorted by time. The whole engine is this loop:

```
while the future-event list is not empty:
    pop the earliest event
    if its time >= the horizon: stop
    jump the clock to that time
    apply the event's state change
    dispatch()          <-- the ONLY place the scheduling policy acts
```

dispatch() asks the current policy: given the doctors who are free right now and the three queues as they currently stand, who starts service? Every policy — IFP, ALT, SBP, WSEPT, Gc·μ — is nothing more than a different answer to that one function. Swapping a policy changes no other line of the simulator, which is exactly why the comparison is fair.

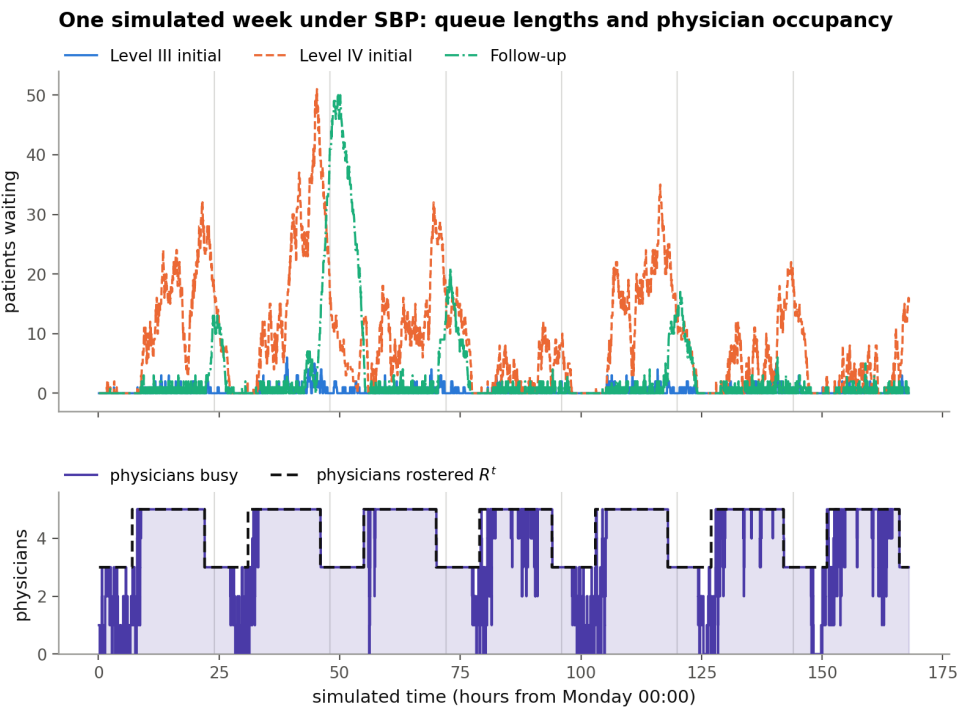
4.2 The five events

Event	What it means	What happens next
SHIFT_CHANGE	The number of rostered doctors changes (07:00, 22:00)	Re-dispatch. A shrinking shift never interrupts anyone mid-consultation; it just blocks new starts.
ARRIVAL	A patient walks in	They join the Level III or Level IV initial queue; re-dispatch.
CONSULT_END	A doctor finishes	The doctor is released. The patient either enters diagnostics, or is discharged; re-dispatch.
EXAM_DEVICE_FREE	A machine finishes occupying a patient	The machine's own waiting queue is pulled, and the patient moves to their next scan.
EXAM_RESULTS_READY	The last of a patient's results comes back	The patient joins the follow-up queue; re-dispatch.

4.3 The module map (mirrors the paper's Figure 2)

The paper describes a five-module architecture. We kept the same five modules with the same names, so the code can be read alongside the paper.

Paper's module	Our file	Holds
Entity Definition	src/entities.py	Patient, ExamOrder, the event-tuple format, the care stages
Event Scheduling	src/engine.py	The heapq future-event list and the loop above
Resource Scheduling	src/resources.py	PhysicianPool (time-varying roster, non-preemptive) and DeviceBank (four machines with FIFO queues)
Strategy Execution	src/policies.py	All five policies — one <code>allocate()</code> method each
Statistical Output	src/metrics.py	Turns a finished run into the paper's KPIs (its equations 24–30)



One simulated week under SBP. Top: how many patients are waiting in each of the three queues. Bottom: doctors busy versus doctors rostered. The daily rhythm and the 07:00/22:00 shift steps are both clearly visible, and the Level IV queue (orange) is the one that builds up. Produced by `run_00_demo.py`.

5. The methods, explained

This is a numerical-methods course, so every technique below is implemented **from scratch**. SciPy appears only for distribution quantiles, the t-distribution, and as a deliberate cross-check of our own optimiser.

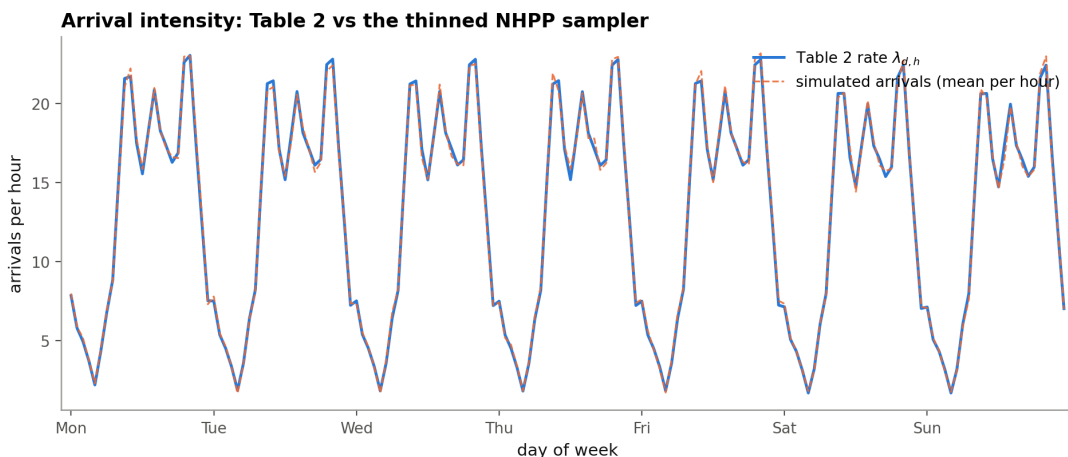
5.1 Generating arrivals: the thinning algorithm

Problem. Patients do not arrive at a constant rate. The rate $\lambda(t)$ changes every hour and repeats weekly — 23 patients/hour on Monday evening, 1.7 on Sunday at 4 a.m. Sampling such a process exactly by inverting its cumulative rate would mean inverting a 168-piece function; easy to get subtly wrong.

Solution (Lewis & Shedler, 1979). Generate too many arrivals at a constant rate, then throw some away:

- 1 Pick λ^* = the highest rate anywhere in the week (23.05/hour, Monday 20:00–21:00).
- 2 Generate a constant-rate process at λ^* by adding up $\text{Exponential}(1/\lambda^*)$ gaps.
- 3 **Keep** each candidate arrival at time t with probability $\lambda(t) / \lambda^*$, and discard it otherwise.

Why it is exact. In a tiny interval the candidate process produces a point with probability $\lambda^* \cdot dt$, and it survives with probability $\lambda(t)/\lambda^*$. The product is $\lambda(t) \cdot dt$ — which is the definition of the process we wanted. The acceptance rate here is about 58%, so roughly 1.7 candidates per kept arrival. `src/rng.py`



Proof that the sampler works: the blue line is the paper's Table 2 rate; the orange dashed line is what 200 simulated weeks actually produced. Relative error 1.67%, correlation 0.9991.

5.2 Generating service times: inverse-CDF sampling

Consultation times follow a **truncated exponential**: exponential in shape, but forced to lie between 5 and 15 minutes (initial) or 5 and 25 (follow-up). We sample by inverting the CDF:

$$F(x) = (e^{(-\lambda a)} - e^{(-\lambda x)}) / (e^{(-\lambda a)} - e^{(-\lambda b)})$$

set $F(x) = u$ and solve:

$$x = -(1/\lambda) * \ln(e^{(-\lambda a)} - u * (e^{(-\lambda a)} - e^{(-\lambda b)}))$$

Why not rejection sampling? Because inverse-CDF uses **exactly one random number per service time, always**. Rejection sampling would use a variable number, and that would destroy the synchronisation that common random numbers (§5.7) depends on. This is a deliberate design choice, not an accident.

Consultation	Rate λ	Bounds	Nominal mean	Realised mean
Initial	1/9	[5, 15] min	9 min	9.0926 min
Follow-up	1/15	[5, 25] min	15 min	12.8410 min

A subtlety worth knowing for the viva

The follow-up mean is 12.84, not the 15 the paper states. That is not a bug: a truncated exponential on [5, 25] **cannot** have mean 15 — 15 is its supremum, approached only as $\lambda \rightarrow 0$. So $\lambda = 1/15$ is the only consistent reading of the paper, and the resulting system load $\rho \approx 0.87$ is what reproduces the paper's ~85% doctor utilisation. We have the closed-form mean $E[X] = a + 1/\lambda - (b-a)e^{-\lambda(b-a)} / (1 - e^{-\lambda(b-a)})$ and check it against 200,000 samples in the test suite.

5.3 The five scheduling policies

Each is one method: *given the free doctors and the three queues, return who starts service*. R is the number of free doctors.

Policy	Origin	The rule
IFP	paper, eqs. 8–10	Fill from Level III initial, then Level IV initial, then follow-ups, until the doctors run out.
ALT	paper, eqs. 11–13	At most $\lfloor R/2 \rfloor$ doctors may be on initial consultations at once; the rest take follow-ups. Unused capacity is handed back so nobody idles.
SBP	paper, eqs. 14–23	Mark patient i urgent when their wait $w_i(t) \geq T_\ell - k_\ell$. Serve urgent L3, then urgent L4, then follow-ups, then everyone else.
WSEPT	ours (WP2)	The $c \cdot \mu$ rule: always serve the non-empty class with the largest index $c_\ell \cdot \mu_\ell$.
Gc·μ	ours (WP2)	The generalised $c \cdot \mu$ rule: index $2 \cdot w_{\text{head}} \cdot \mu_\ell / T_\ell^2$, which grows with how long the front patient has already waited.

5.4 The $c \cdot \mu$ rule, and why it is special

The theorem (Cox & Smith 1961). If each waiting class ℓ costs you c_ℓ per minute of delay, and class ℓ takes on average $1/\mu_\ell$ minutes to serve, then the policy that minimises total delay cost is: **always serve the class with the largest $c_\ell \cdot \mu_\ell$** . It is not a heuristic — it is provably optimal. This is what makes it a genuinely different kind of answer from the paper's three hand-designed rules.

We set $c_\ell = 1/T_\ell$ — *a minute of waiting hurts in inverse proportion to how much waiting that class can tolerate* — which turns the triage targets straight into delay costs. With the paper's numbers:

Class	T_ℓ (min)	$E[\text{service}]$ (min)	μ_ℓ	$c_\ell = 1/T_\ell$	index $c \cdot \mu$
Level III initial	30	9.093	0.10998	0.033333	0.003666
Level IV initial	120	9.093	0.10998	0.008333	0.000916
Follow-up	60	12.841	0.07788	0.016667	0.001298

Read the index column carefully — it is a real finding

The indices are **constants**, so the classical $c \cdot \mu$ rule collapses to a fixed priority order: **Level III initial > follow-up > Level IV initial**. That is a genuine property of the theorem, not a shortcut in our code, and it is precisely the limitation the *generalised* $c \cdot \mu$ rule fixes by making the index grow with the head-of-line patient's delay. Gc· μ is delay-adaptive like SBP — but it is derived, and it has **no parameters to tune at all**.

How we judge it fairly

The $c\cdot\mu$ rule is optimal for the *holding cost*, not for the plain average wait. Reporting only the plain average would be a category error. So `src/metrics.py` also computes **holding cost = mean of $w_i / T_{\text{level}(i)}$** — waiting measured in units of each patient's own tolerance — and we rank on that too.

5.5 Optimising a noisy function

Step 1 — make the objective deterministic

A simulation gives a different answer every time you run it. Simplex methods stall on noisy surfaces because they cannot tell a real improvement from luck. Fix: **sample average approximation with common random numbers**. Every candidate (k_1, k_2) is scored on the *same fixed set* of 50 simulated weeks. Evaluating the same point twice now returns the *identical* number, so the optimiser is minimising a genuine deterministic function.

Step 2 — handle the constraint with a penalty

The real problem is constrained: minimise waiting *subject to* the service level staying above 95%. Nelder-Mead is unconstrained, so we fold the constraint into the objective with an **exterior penalty**:

```
F(k1, k2) = W_total + mu * SUM over levels of max(0, SL_min - SL_level)

mu = 500 minutes per unit of service-level shortfall
    -> one percentage point of shortfall costs as much as 5 extra
        minutes of average waiting.
```

Large enough that an infeasible point is never optimal; small enough that the surface stays smooth near the boundary instead of becoming a cliff the simplex bounces off. Box bounds ($k \geq 0$) are handled by **projection** — clipping each trial point back into the box — so the simplex can slide *along* a bound rather than being pushed away from it.

Step 3 — the Nelder-Mead simplex, written out

For 2 variables the simplex is a triangle with 3 corners. Each iteration sorts the corners best-to-worst and tries, in order:

Reflect	Flip the worst corner through the middle of the others. ($\alpha = 1$)
Expand	If that reflection is the new best, push twice as far — we found a good direction. ($\gamma = 2$)
Contract	If it is no better than the second-worst, pull halfway back instead. ($\rho = 0.5$)
Shrink	If even that fails, pull every corner halfway toward the best one. ($\sigma = 0.5$)
Stop	When the triangle is smaller than 0.05 in every coordinate <i>and</i> the spread of its values is under 0.02.

Two practical details that matter. **Per-dimension step size:** k_1 lives on roughly $[0, 30]$ while k_2 lives on $[0, 120]$, so we build the starting triangle with steps (4, 15) rather than one scalar — otherwise the simplex wastes iterations just re-scaling itself. **Multi-start:** Nelder-Mead only finds a *local* minimum, so we run it from four dispersed starting points; all four landing on the same value is evidence the surface is unimodal here. `src/optimizers.py`

Making the benchmark honest

Both methods minimise the *same* objective on the *same* simulated weeks, and the budget is counted in **week-long simulations, not seconds**, so the comparison does not depend on the machine or on how many cores we had. One trap we avoid: the two methods use different replication counts per point, so their raw objective values are estimates of the same thing with different precision and are **not** directly comparable. The script therefore re-scores both optima on one identical full-length batch before declaring anything.

5.6 Common random numbers (the WP5 idea)

The identity everything rests on. For any two policies A and B:

$$\text{Var}(A - B) = \text{Var}(A) + \text{Var}(B) - 2 * \text{Cov}(A, B)$$

With independent random streams, $\text{Cov} = 0$ and the variance of the difference is just the sum. But if both policies face the **identical simulated week** — the same patients, the same arrival times, the same service durations, the same scans — then a busy week inflates *both* policies together. Cov becomes strongly positive, and the variance of the **difference** collapses. Note that each policy's own variance is unchanged: CRN sharpens *comparisons*, not individual estimates. That distinction is the whole point.

Why ours is near-perfect

Most simulation code loses most of the CRN benefit through **desynchronisation**: once two policies make different decisions they start consuming random numbers in a different order, and patient #1723 ends up with different service times in the two runs. We avoid this structurally, by three design choices:

- **Six independent substreams** (arrivals, triage levels, initial service, follow-up service, exam need, exam choice), so a change in how many draws one dimension makes cannot shift another.
- **The entire patient list is generated before the run starts.** The engine never draws a random number while simulating — so patient #1723 is bit-for-bit identical under every policy.
- **One uniform per random variate** (§5.2), so no single variate can knock the stream out of step.

The measured result: policy-to-policy correlations of +0.996 to +0.9997, and **99.0% of the comparison variance removed**. `src/rng.py`

5.7 The cost model (the WP4 idea)

The paper's staffing section shows that more doctors means shorter waits. True, but useless as advice — it has no stopping point. To get an actual recommendation you need to price the trade-off:

```
C(R, k1, k2) = alpha * physician-hours per week
              + beta  * total patient waiting-minutes per week
              + gamma * number of SLA violations per week

alpha = 120   an ED attending on ~USD 250k/yr loaded over ~2080 h/yr
beta  = 0.50  one patient-HOUR of waiting is worth 30 units
gamma = 50    a breach costs far more than the minutes that caused it
```

R is the integer roster (morning, afternoon, night). The search is two stages, because running a full optimiser inside all 140 rosters would be wasteful: **screen** every roster once cheaply and drop the unstable ones (where more than 15% of the week's patients are still in the system at the end — their queues are growing, not clearing, so their mean wait is meaningless), then **refine** the best few with a full Nelder-Mead re-tune of (k_1, k_2) .

Why the thresholds are re-tuned inside every roster

The best (k_1, k_2) for a 3-doctor night shift is *not* the best for a 5-doctor one. Holding them fixed while varying staffing would score every alternative roster at settings chosen for a different one. That is what makes this a **joint** optimisation rather than a staffing sweep — and the measured threshold drift confirms it matters: the optimum moves from (2.31, 8.37) at (5,5,3) to (12.20, 23.40) at (6,6,3).

5.8 The statistics

Tool	What it does	Why this one
Paired t-test	$t = \text{mean}(d) / (\text{sd}(d)/\sqrt{n})$ on the per-replication differences $d = a - b$	The design <i>is</i> paired, because CRN gives both policies the same weeks. An unpaired test would throw away exactly the variance reduction CRN bought.
Bonferroni correction	Test each comparison at α/m instead of α	With 6 policies there are 15 comparisons; without correction you would expect a false positive by chance. $\alpha_{\text{adj}} = 0.05/15 = 0.00333$.
Cohen's d (paired)	$d = \text{mean}(\text{differences}) / \text{sd}(\text{differences})$	Statistical significance only says 'not zero'. Effect size says 'how much', which is what a hospital actually cares about.
reps_for_halfwidth	Smallest n with $t_{0.975, n-1} \cdot \text{sd}/\sqrt{n} \leq h$	Turns the variance reduction into the practical currency: how many runs you can skip. Solved by iteration because the t-quantile itself depends on n .

6. Project map: which file does what

```

project/
  README.md          <- start here
  healthcare-14-00099-v2.pdf <- the base paper
  Updated_Project_Proposal.md
  edsim/
    README.md        <- the full written guide
    requirements.txt
  docs/
    PROJECT_GUIDE.pdf <- THIS DOCUMENT
    PAPER_MAPPING.md  <- every paper equation -> the code
    METHODS.md        <- the numerical methods in full
    make_guide.py     <- generates this PDF
  src/                (15 modules) <- the library; never run directly
  scripts/            (10 scripts) <- the runnable experiments
  tests/test_model.py <- 33 correctness checks
  results/            <- EVERYTHING the scripts produce
    RESULTS_SUMMARY.md <- the headline tables
    PROVENANCE.md      <- which command made which file, when
  tables/             (32 x .csv + .md)
  figures/             (22 x .png + .pdf)
  raw/                (per-replication data + JSON manifests)

```

6.1 The library, src/

File	Responsible for
config.py	Every number in the model, in one place. The 7×24 arrival table, triage mix, service bounds, the four machines, shift rosters, and the paper's published results used as validation targets. Each constant is annotated with the paper section it comes from; anything the paper does not give is marked OUR CHOICE with a reason.
entities.py	Patient, ExamOrder, the event tuple, the care stages.
distributions.py	Inverse-CDF samplers (truncated exponential and lognormal) and the closed-form truncated mean.
rng.py	Thinning for the arrivals, the pre-generated patient stream, and the CRN / independent-streams switch.
resources.py	Doctors (time-varying roster) and the four machines.
policies.py	All five scheduling policies.
engine.py	The discrete-event loop.
metrics.py	The paper's KPIs, plus a written explanation of which reading of 'average waiting time' the paper's numbers actually imply.
experiment.py	Runs N replications over a reused process pool; caches patient streams per worker; the batch evaluator both optimisers share.
stats.py	t-tests, Bonferroni, Cohen's d, the CRN variance analysis.
optimizers.py	The hand-written Nelder-Mead, the grid search, and the penalised objective.
staffing.py	The weekly cost function and the roster search.
plotting.py	One shared visual language for all 22 figures.
console.py	Banners, progress bars, ASCII tables — the live output.
report.py	The provenance machinery: every table gets a header naming the command that produced it, and every run is logged to PROVENANCE.md.

6.2 The experiments, scripts/

Each one prints a full narrative to the terminal *and* writes its tables and figures. Runtimes are for a 16-core machine.

#	Script	What it does	Time
0	run_00_demo.py	Watch the engine work. Live event trace, then a console dashboard of queue lengths updating every simulated hour, then all five policies on one shared week.	15 s
1	run_01_validate_paper.py	WP1. Checks the arrival sampler against Table 2 and the service sampler against its closed form, then runs 100 replications and compares every published KPI, t-test, effect size and equipment number.	25 s
2	run_02_grid_search.py	Reproduces the paper's two-stage brute-force search and records what it cost. Deliberately the expensive one — 50,415 week-long simulations — because that expense is the WP3 result.	10 min
3	run_03_wsept_policy.py	WP2. Adds the two c-μ policies, prints the index values the theory produces, and sweeps the one weight the paper does not supply.	20 s
4	run_04_nelder_mead.py	WP3. Our simplex against the same objective, with a live iteration trace, a 4-start check, a SciPy cross-check, and the budget comparison.	3 min
5	run_05_staffing_cost.py	WP4. Screens 140 rosters, re-tunes the thresholds inside the best ones, prices the paper's own scenarios, and stress-tests the answer against the cost weights.	3 min
6	run_06_crn_variance.py	WP5. Runs the whole comparison twice — once with CRN, once with independent streams — and quantifies what CRN buys.	20 s
7	run_07_sensitivity.py	Reproduces the paper's robustness study, extended to five policies.	2.5 min
8	run_08_final_comparison.py	The consolidated master table, the full significance matrix, the verdict under three objectives, and RESULTS_SUMMARY.md.	15 s
—	run_all.py	Runs 1–8 in dependency order with a summary table at the end.	20 min

6.3 Where results go, and how to trust them

Every table is written twice: a **.csv** for machines and a **.md** for pasting into the report. The CSV carries a four-line provenance header:

```
# produced by: scripts/run_01_validate_paper.py
# command    : python scripts/run_01_validate_paper.py --reps 100 --seed 20250402
# timestamp   : 2026-08-28 21:21:16
# args       : {"reps": 100, "seed": 20250402, "workers": 14, ...}
```

The same information — plus every output path and every key number — is appended to **results/PROVENANCE.md** and written as JSON to **results/raw/<script>.manifest.json**. So for any file in results/ you can answer *'which command produced this, when, with what settings?'* by reading one file. Results are deterministic given the seed: two runs with the same flags produce byte-identical tables.

7. How to run everything

7.1 First time

```
cd "c:\Buet\CSE 402\project\edsim"
pip install -r requirements.txt

python tests/test_model.py          # 33 correctness checks    (~20 s)
python scripts/run_00_demo.py      # watch the simulator run (~15 s)
python scripts/run_all.py --quick  # whole study, fast pass  (~5 min)
python scripts/run_all.py          # whole study, full      (~20 min)
```

Run the demo first

`run_00_demo.py` is the fastest way to convince yourself (or an examiner) that the engine really does what the paper describes. It prints every state transition — arrival, dispatch decision, consultation start and end, machine seizure, results returning — and then redraws a live dashboard of the three queue lengths and doctor occupancy once per simulated hour.

7.2 Flags every script accepts

Flag	Default	Meaning
<code>--reps N</code>	100	Replications per policy (the paper's setting)
<code>--seed S</code>	20250402	Base seed — fixes every result reproducibly
<code>--workers W</code>	cores – 2	Parallel worker processes
<code>--quick</code>	off	Fast low-fidelity pass for smoke-testing the pipeline
<code>--no-figures</code>	off	Skip figure rendering

7.3 Useful specific runs

```
# watch a different policy for 6 simulated hours, slowed down
python scripts/run_00_demo.py --policy SBP --trace-hours 6 --speed 0.05

# a cheaper grid search (the expensive script)
python scripts/run_02_grid_search.py --coarse-reps 10 --fine-reps 30

# try your own cost assumptions for the staffing recommendation
python scripts/run_05_staffing_cost.py --alpha 150 --beta 1.0 --gamma 100

# re-run only two stages of the pipeline, with more replications
python scripts/run_all.py --only 4,8 --reps 200

# regenerate this PDF after a fresh run
python docs/make_guide.py
```

7.4 Reading the terminal output

Every script prints in the same shape: a banner, the exact settings in force, then numbered STEPs. Lines marked **[OK]** and **[!!]** are *automated assertions about the results, evaluated live* — they are not decoration. If a structural finding of the paper fails to reproduce, the script says so out loud rather than staying silent. Progress bars show elapsed time and ETA, and the final block lists every file written.

8. Results and analysis

All numbers below come from `python scripts/run_all.py` at seed 20250402 with 100 replications per policy.

8.1 WP1 — does the rebuild match the paper?

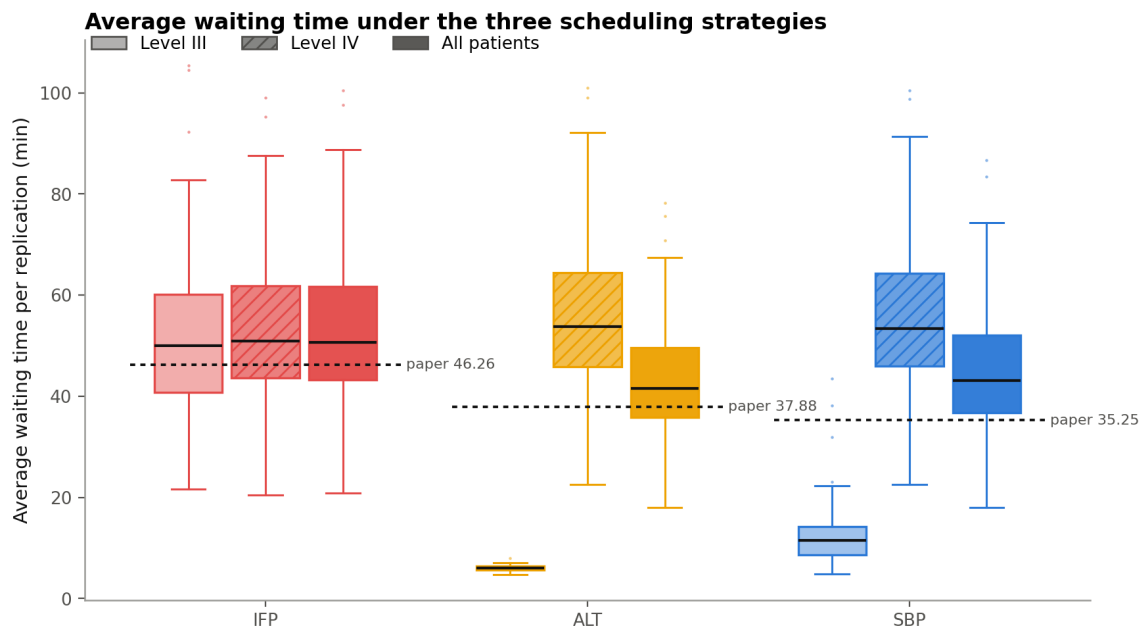
Policy	Metric	Ours	sd	Paper	Deviation
IFP	Mean waiting time, all patients (min)	52.43	14.10	46.26	+13.3%
IFP	Mean waiting time, Level III (min)	50.88	14.81	44.14	+15.3%
IFP	Mean waiting time, Level IV (min)	52.94	13.98	not published	n/a
IFP	Service level, Level III (%)	100.00	0.00	100.00	+0.0%
IFP	Service level, Level IV (%)	100.00	0.00	100.00	+0.0%
IFP	Physician utilisation (%)	86.36	1.92	85.43	+1.1%
ALT	Mean waiting time, all patients (min)	42.39	10.70	37.88	+11.9%
ALT	Mean waiting time, Level III (min)	6.05	0.57	6.82	-11.2%
ALT	Mean waiting time, Level IV (min)	54.61	14.10	not published	n/a
ALT	Service level, Level III (%)	99.96	0.13	98.08	+1.9%
ALT	Service level, Level IV (%)	87.37	5.59	89.29	-2.2%
ALT	Physician utilisation (%)	86.34	1.93	85.41	+1.1%
SBP	Mean waiting time, all patients (min)	44.08	11.91	35.25	+25.1%
SBP	Mean waiting time, Level III (min)	12.35	6.04	10.48	+17.9%
SBP	Mean waiting time, Level IV (min)	54.74	14.07	43.89	+24.7%
SBP	Service level, Level III (%)	99.99	0.04	100.00	-0.0%
SBP	Service level, Level IV (%)	89.40	4.51	100.00	-10.6%
SBP	Physician utilisation (%)	86.34	1.93	84.98	+1.6%

What matches almost exactly: both service levels under IFP (100%/100%), ALT's Level III waiting time (6.05 vs 6.82 published), SBP's Level III waiting time (12.35 vs 10.48), doctor utilisation (86.3% vs 85.0–85.4%), and — strikingly — the equipment table below.

What runs high: the absolute waiting times, by 12–25%. The explanation is a single number: the published parameters imply a system load of $\rho \approx 0.87$, and near saturation the mean wait is hypersensitive to the last percentage point of utilisation. The paper's underlying dataset is not public, so an exact match is not attainable — and we do not claim one.

Equipment	Exams/week	Our utilisation	Paper's	Mean machine queueing delay
LABORATORY	1248	14.73%	14.36%	0.074 min
XRAY	394	15.59%	15.20%	0.427 min
CT	748	18.18%	17.64%	0.287 min
ULTRASOUND	297	19.41%	19.03%	0.977 min

Equipment utilisation, reproduced within ~ 0.5 percentage points on all four machines. Every machine sits far below the doctors' 86%, which reproduces the paper's central managerial conclusion: **doctors, not equipment, are the bottleneck** — so buying another CT scanner would not help.



Our reproduction of the paper's Figure 3. Three boxes per policy: Level III, Level IV, all patients. Dashed lines mark the paper's published means. Note ALT's tiny Level III box (left) — it sprints on urgent patients — and its tall Level IV box, which is the cost.

8.2 WP2 — does the $c\cdot\mu$ rule deliver?

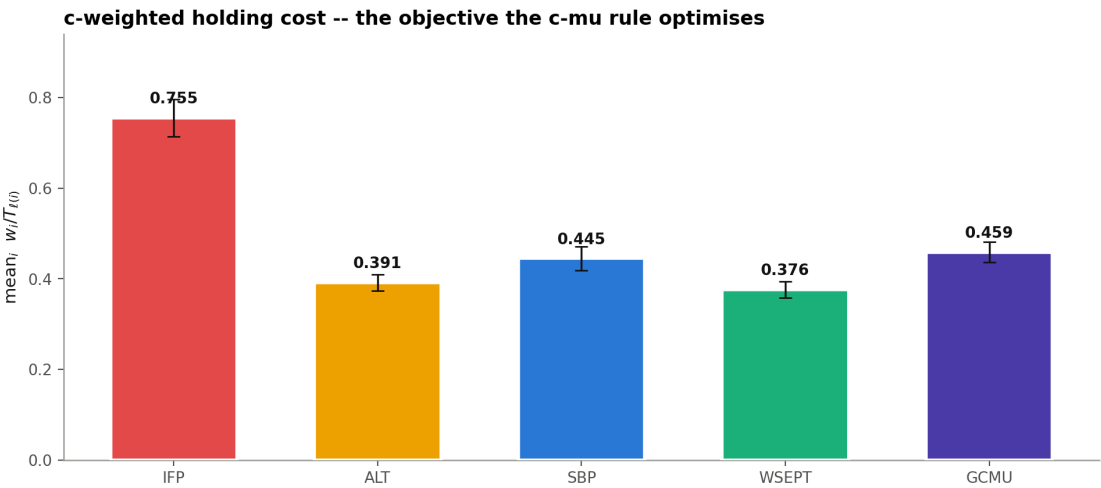
Policy	Mean wait	95% CI	Level III	Level IV	Holding cost	SL3	SL4
IFP	52.43	[49.63, 55.22]	50.88	52.94	0.7550	100.00 %	100.00%
ALT	42.39	[40.26, 44.51]	6.05	54.61	0.3913	99.96%	87.37%
SBP	44.08	[41.72, 46.44]	12.35	54.74	0.4450	99.99%	89.40%
SBP*	44.18	[41.81, 46.55]	14.14	54.27	0.4570	95.42%	95.64%
WSEPT	42.32	[40.21, 44.43]	3.76	55.30	0.3763	100.00 %	86.87%
GCMU	44.05	[41.81, 46.29]	14.59	53.94	0.4588	100.00 %	90.99%

WSEPT wins on its own objective, as the theorem predicts — holding cost 0.3763 against IFP's 0.7550, a 50% reduction — and, on this system, it also happens to win on the plain unweighted average (42.32 min). Every pairwise difference against the paper's three heuristics is significant after Bonferroni correction.

But look at the service-level columns. WSEPT reaches only 86.9% on Level IV. The rules that sprint hardest on Level III are exactly the rules that let Level IV breach. This is the trade-off surface, not a league table — and it is why §8.7 gives three answers rather than one.

An honest caveat we state in the code as well

The Cox-Smith optimality proof assumes a *single-stage* multi-class queue. Our model has **re-entrant flow** (patients come back for a follow-up after diagnostics) and a **time-varying** pool of doctors, so the guarantee is only approximate here. The script says so out loud if the empirical ranking ever contradicts the theory.



The objective the $c\cdot\mu$ rule is provably optimal for: waiting measured in units of each patient's own clinical tolerance. Lower is better. WSEPT wins; IFP is twice as expensive as anything else.

8.3 WP3 — optimiser versus brute force

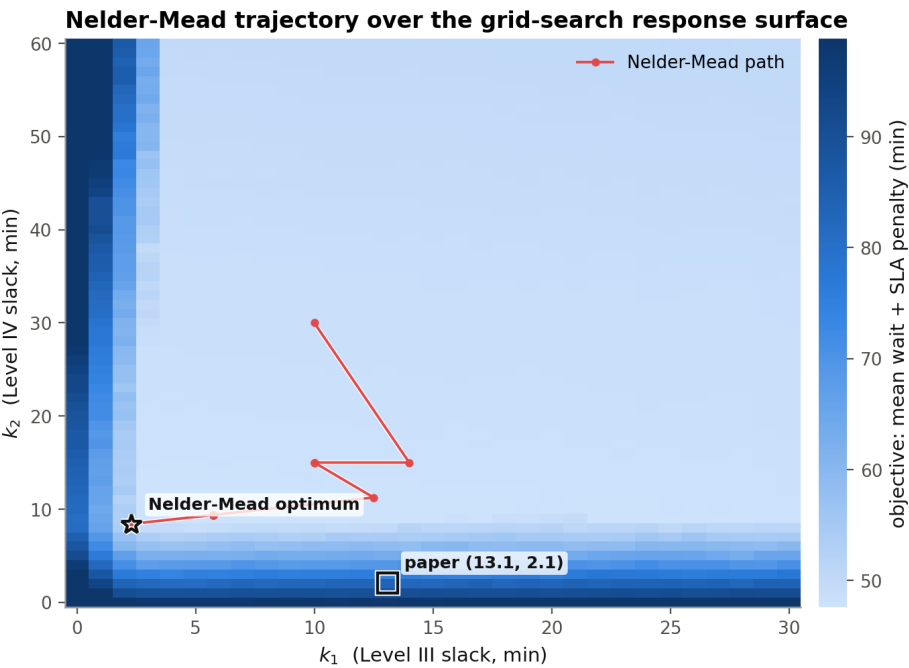
Method	Points	Reps each	Week-long simulations	Best objective
Grid search, coarse stage (paper's method)	1,891	15	28,365	47.6105
Grid search, fine stage (paper's method)	441	50	22,050	46.4601
Grid search, TOTAL	2,332	-	50,415	46.4601
Nelder-Mead, single start (ours)	53	50	2,650	46.4668
Nelder-Mead, 4 starts (ours)	196	50	9,800	46.4668

19× fewer simulations, single start	0.006 minutes worse than the lattice optimum	4 / 4 starts converging to the same objective	0.0000 difference from SciPy's implementation
---	--	---	---

The grid search spent **50,415 week-long simulations** — about 970 simulated patient-years of emergency department operation — to find (4.10, 8.20). Our simplex found an equally good point in **2,650**. Scored on one identical full-length batch afterwards, the two optima differ by **0.006 minutes** — far below the noise floor of the simulation itself.

Setting	(k ₁ , k ₂)	Mean wait	sd	SL3	SL4	Constraint
Nelder-Mead optimum	(2.31, 8.37)	44.18	11.95	95.42%	95.64%	feasible
grid-search optimum	(4.10, 8.20)	44.17	11.94	97.73%	95.49%	feasible
paper optimum (13.1, 2.1)	(13.10, 2.10)	44.08	11.91	99.99%	89.40%	INFEASIBLE

All three optima re-scored on identical weeks. The bottom row is the finding: **the paper's own published thresholds are infeasible on our system**, because k₂ = 2.1 only rescues a Level IV patient after 117.9 of their 120 minutes.



The simplex trajectory drawn on the very response surface the lattice had to evaluate in full. The dark band along the bottom is the SLA penalty; the paper's point (square) sits inside it, while the simplex walks down from (10, 30) to the feasible optimum (star).

8.4 WP4 — how many doctors should the ED roster?

Configuration	(k_1 , k_2)	Doctor-h	Mean wait	SL3	SL4	Staff cost	Wait cost	Total / week
paper's fixed roster (5,5,3)	(2.31, 8.37)	714	44.18 min	95.42%	95.64%	85,680	48,798	139,391
cost-optimal roster (6, 6, 3)	(12.20, 23.40)	819	10.97 min	99.99%	99.93%	98,280	12,174	110,513

The recommendation: roster (6, 6, 3) — one more doctor on each day shift — **with thresholds re-tuned to (12.20, 23.40)**. That costs 105 more doctor-hours a week, but it buys back 33 minutes of average waiting per patient and cuts the SLA penalty from 4,913 to 60 cost units — a net weekly saving of **28,878 cost units (20.7%)**.

Scenario	Roster	Doctor-h	Mean wait	Util.	SL4	Staff cost	Wait cost	Total
S0	(5, 5, 3)	714	44.18 min	86.3%	95.64%	85,680	48,798	139,391
S1	(5, 5, 4)	777	35.19 min	79.5%	98.53%	93,240	38,959	134,121
S2	(5, 5, 5)	840	31.25 min	73.7%	99.32%	100,800	34,659	136,490
S3	(5, 6, 5)	889	16.34 min	69.8%	99.98%	106,680	18,166	124,942
S4	(5, 7, 5)	938	11.45 min	66.1%	100.00%	112,560	12,722	125,327
S5	(6, 7, 5)	994	4.32 min	62.4%	100.00%	119,280	4,800	124,082
S6	(7, 7, 5)	1050	2.38 min	59.1%	100.00%	126,000	2,650	128,652

The paper's own seven staffing scenarios, now with a price attached. The paper stops at '*more staff lowers waiting*', which has no stopping point. With a cost function the sequence has a clear **interior optimum** at S5 — and our free search over all 140 rosters finds a cheaper one still, because Table 10 only walks one particular path through the roster space.

Cost scenario	α	β	γ	Winning roster	Its cost	Cost of (5,5,3)	Saving
baseline	120	0.5	50	(6, 6, 3)	110,513	139,391	-20.7%
waiting valued 4x	120	2	50	(6, 7, 3)	133,862	285,787	-53.2%
waiting valued 1/4	120	0.125	50	(5, 6, 3)	97,408	102,792	-5.2%
physicians 1.5x dearer	180	0.5	50	(6, 6, 3)	159,653	182,231	-12.4%
SLA breach 10x dearer	120	0.5	500	(6, 6, 3)	111,049	183,605	-39.5%

The honest reading of the sensitivity table

The winning roster **changes** with the cost weights — (6,6,3) at the baseline, (6,7,3) if waiting is valued four times higher, (5,6,3) if it is valued a quarter as much. That is not a weakness of the method; it *is* the finding. The right staffing level genuinely depends on how the hospital prices a patient's waiting minute against a doctor's salary, so **those weights have to be agreed before the roster can be set**. What does not change: every scenario beats the paper's fixed (5,5,3), by between 5% and 53%.

8.5 WP5 — what do common random numbers buy?

Comparison	Var (independent)	Var (CRN)	Variance removed	Efficiency	CI ½-width (ind)	CI ½-width (CRN)
IFP vs ALT	241.3350	12.4395	94.85%	19.4x	3.0825	0.6998
IFP vs SBP	330.4789	6.1206	98.15%	54.0x	3.6071	0.4909
IFP vs WSEPT	288.2034	13.1582	95.43%	21.9x	3.3685	0.7198
IFP vs GCMU	216.1898	8.5023	96.07%	25.4x	2.9175	0.5786
ALT vs SBP	220.7672	2.1457	99.03%	102.9x	2.9482	0.2907
ALT vs WSEPT	191.6437	0.1071	99.94%	1789.3x	2.7469	0.0649
ALT vs GCMU	223.7516	0.4610	99.79%	485.4x	2.9681	0.1347
SBP vs WSEPT	198.0362	2.2323	98.87%	88.7x	2.7923	0.2965
SBP vs GCMU	268.9685	1.0273	99.62%	261.8x	3.2542	0.2011
WSEPT vs GCMU	215.2730	0.5935	99.72%	362.7x	2.9113	0.1529

99.0% median variance removed	96x median efficiency gain	+0.996 to +0.9997 induced correlation	20-30 CRN reps replace the paper's 100
----------------------------------	-------------------------------	--	---

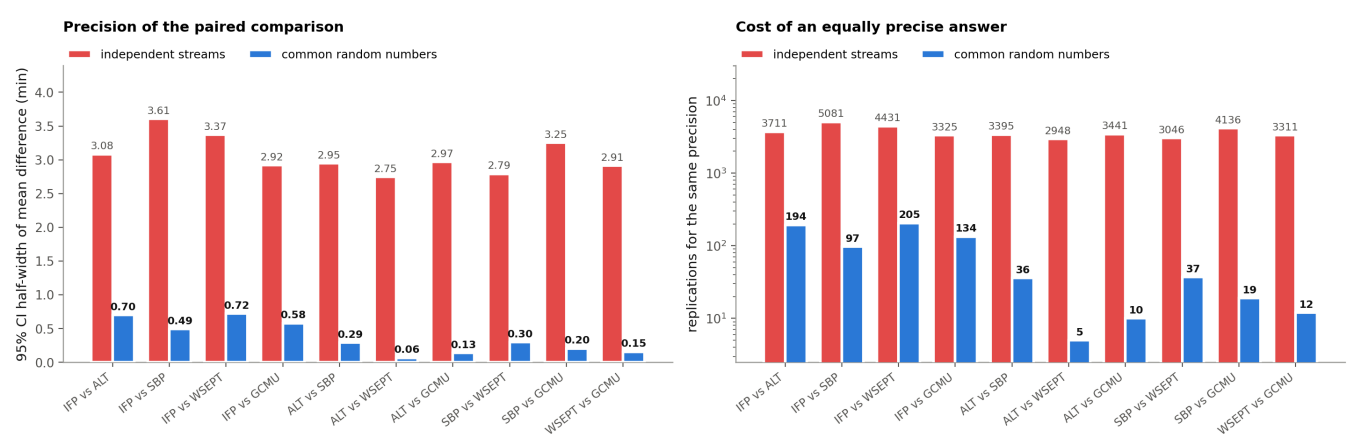
Two results worth stating separately. First, **CRN does not move the point estimates** — each policy's own mean and standard deviation are unchanged between the two arms. It changes the *dependence* between runs, not the marginal law of any single one, which is why it is a variance-reduction technique and not a bias.

Second, **it changes conclusions**. With independent streams, several comparisons fail to reach the Bonferroni threshold; with CRN the same comparisons are decisive. Where the two arms disagree, CRN is the arm to believe: it is testing the same hypothesis with a lower-variance estimator of the same quantity.

Do not over-claim this one

A 96x efficiency gain does **not** mean you should run one replication. The ratio is itself estimated from the data, and a t-interval needs enough degrees of freedom to be trustworthy. The practical reading is that **20-30 CRN replications give comparisons at least as sharp as the paper's 100 independent ones** — a 3-5x saving that is safe to actually take.

What Common Random Numbers buy (target CI half-width 0.5 min)

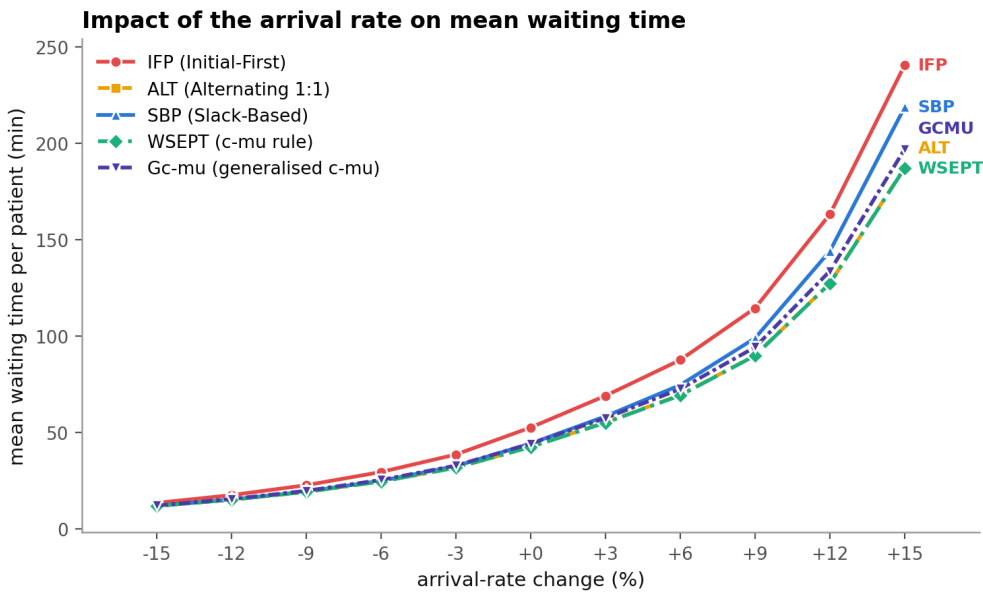


Left: the confidence interval on each pairwise difference shrinks by roughly 4-40x. Right: the number of replications needed to pin that difference to ±0.5 min, log scale — thousands become dozens.

8.6 Robustness — do the conclusions survive?

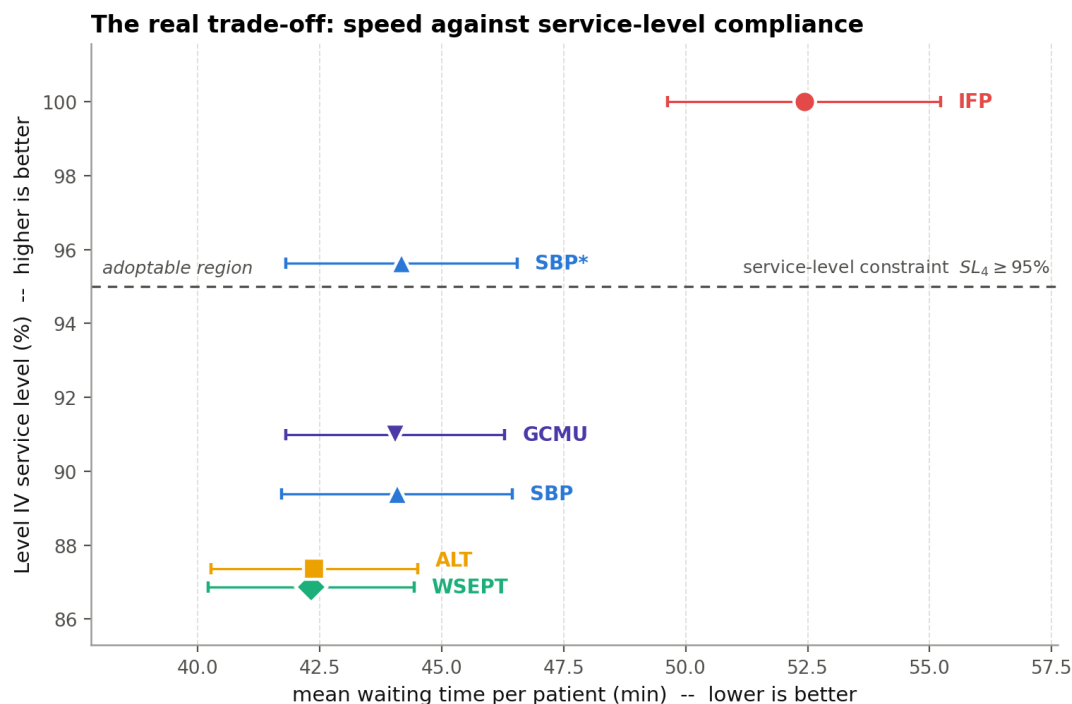
Test	What we varied	Result
Arrival rate	Every hourly rate scaled –15% to +15% in 3% steps (p from 0.74 to 1.00)	Policies are nearly interchangeable under light load and fan out sharply as the system saturates — policy spread grows from 1.5 min to 61 min. The paper's Figure 4 finding, reproduced.
Staffing	The paper's seven rosters S0–S6	Waits fall with clear diminishing returns, and the <i>choice of policy</i> matters most exactly when doctors are scarce. Policy spread falls from 10.4 min at S0 to 0.07 min at S6. Figure 5 reproduced.
Service-time distribution	Truncated exponential → truncated lognormal ($\sigma = 0.3$, same nominal means)	All waits rise 44–54% because of the heavier right tail, but the ranking is identical . The conclusions are not an artefact of assuming exponential service times — as the paper also reports.

Policy	TruncExp wait	Lognormal wait	Change	Relative	SL4 (exp)	SL4 (lognormal)
IFP	52.43 min	75.43 min	+23.00	+43.9%	100.00%	100.00%
ALT	42.39 min	64.42 min	+22.04	+52.0%	87.37%	76.65%
SBP	44.08 min	67.70 min	+23.62	+53.6%	89.40%	80.57%
WSEPT	42.32 min	64.46 min	+22.14	+52.3%	86.87%	75.69%
GCMU	44.05 min	66.42 min	+22.37	+50.8%	90.99%	82.48%



Reproduction of the paper's Figure 4, extended to all five policies. Under light load (left) the policies are indistinguishable; as the department saturates (right) the choice of rule becomes worth minutes per patient.

8.7 The final verdict



The whole project in one picture. Horizontal: average waiting time (lower is better). Vertical: Level IV service level (higher is better). The dashed line is the 95% constraint — **only policies above it are adoptable**. IFP is safe but slow; ALT, SBP, WSEPT and Gc·μ are fast but infeasible; only SBP*, our re-tuned policy, is both.

Comparison	Mean diff (min)	95% CI	t	p	Sig.	Cohen's d
IFP vs ALT	+10.042	[+8.981, +11.102]	28.471	1.774e-49	Yes	+2.847
IFP vs SBP	+8.346	[+7.601, +9.090]	33.734	4.294e-56	Yes	+3.373
IFP vs SBP*	+8.246	[+7.520, +8.973]	34.140	1.442e-56	Yes	+3.414
IFP vs WSEPT	+10.108	[+9.017, +11.199]	27.865	1.180e-48	Yes	+2.786
IFP vs GCMU	+8.378	[+7.501, +9.255]	28.731	7.928e-50	Yes	+2.873
ALT vs SBP	-1.696	[-2.136, -1.255]	-11.577	4.145e-20	Yes	-1.158
ALT vs SBP*	-1.795	[-2.247, -1.344]	-11.961	6.203e-21	Yes	-1.196
ALT vs WSEPT	+0.066	[-0.032, +0.165]	2.025	0.04553	No	+0.203
ALT vs GCMU	-1.664	[-1.868, -1.460]	-24.507	7.946e-44	Yes	-2.451
SBP vs SBP*	-0.099	[-0.155, -0.044]	-5.372	5.168e-07	Yes	-0.537
SBP vs WSEPT	+1.762	[+1.313, +2.212]	11.794	1.418e-20	Yes	+1.179
SBP vs GCMU	+0.032	[-0.273, +0.337]	0.315	0.75312	No	+0.032
SBP* vs WSEPT	+1.861	[+1.400, +2.323]	12.127	2.743e-21	Yes	+1.213
SBP* vs GCMU	+0.131	[-0.182, +0.445]	1.260	0.21079	No	+0.126
WSEPT vs GCMU	-1.730	[-1.962, -1.498]	-22.458	1.218e-40	Yes	-2.246

All 15 pairwise paired t-tests, Bonferroni-corrected ($\alpha_{adj} = 0.05/15 = 0.00333$). Only three comparisons fail to reach significance, and all three are between policies that genuinely perform the same.

9. Paper vs. ours, side by side

9.1 Where we agree

Finding	Paper	Ours	Verdict
IFP is the worst policy on mean wait	46.26 min	52.43 min	Reproduced
ALT collapses Level III waiting	6.82 min	6.05 min	Reproduced (11% off)
ALT breaks the Level IV service level	89.29%	87.37%	Reproduced (2% off)
IFP holds 100% on both levels	100 / 100	100 / 100	Exact
Doctor utilisation is ~85% and near-identical across policies	84.98–85.43%	86.34–86.36%	Reproduced
Equipment is far from the bottleneck	14.4–19.0%	14.7–19.4%	Reproduced (<0.5 pp)
Policies diverge as load rises	Figure 4	1.5 → 61 min spread	Reproduced
Adding staff has diminishing returns	Figure 5	10.4 → 0.07 min spread	Reproduced
Ranking survives a lognormal service time	Sec 3.4.3	identical ranking	Reproduced

9.2 Where we differ, and why

Difference	Why it happens	Does it undermine anything?
Our waiting times are 12–25% higher	The published parameters imply $\rho \approx 0.87$. Near saturation, mean wait is hypersensitive to the last point of utilisation. The source dataset is not public, so the exact patient mix cannot be recovered.	No. Every <i>relative</i> and structural result reproduces; only the level differs. We report this openly rather than tuning to match.
SBP is not the best policy on our system	The paper's (13.1, 2.1) is optimal for the paper's system, not ours. With $k_2 = 2.1$ a Level IV patient is rescued only at minute 117.9 of 120 — too late under heavier congestion.	No — it is the point. Re-tuning under the constraint (WP3) restores SBP* to the best <i>feasible</i> policy. This is the strongest argument in the project for re-tuning rather than copying published parameters.
The paper's thresholds are infeasible here	They reach 89.4% on Level IV against a 95% target.	It sharpens the contribution. The paper never had to face the constraint because on its system the tuned thresholds happened to satisfy it.

9.3 What we add that the paper does not have

Addition	Where it lives
The $c\cdot\mu$ / WSEPT policy and its generalisation	src/policies.py — CMuPolicy
A holding-cost metric, so the $c\cdot\mu$ rule is judged on its own objective	src/metrics.py — holding_cost
A hand-written Nelder-Mead simplex with projection bounds	src/optimizers.py — nelder_mead()
An exterior penalty for the service-level constraint	src/optimizers.py — SimulationObjective.penalty()
Simulation-budget accounting, so optimisers can be compared fairly	src/optimizers.py, run_04 STEP 4
A weekly cost model and an integer roster search	src/staffing.py, run_05
Exact common random numbers and the variance reduction quantified	src/rng.py, src/stats.py — crn_efficiency()
Provenance logging for every single artefact	src/report.py — Artifact
A 33-check verification suite	tests/test_model.py

10. The judgement calls we had to make

The paper's dataset is not public and a few modelling details are left unspecified. There are exactly five such decisions. All are flagged OUR CHOICE in `src/config.py` with a reason, and each is swept or stress-tested somewhere in the pipeline.

Decision	What we chose	Why, and how we check it
The numeric service-level constraint. The paper writes $SL_t \geq SL_t^{\min}$ but never prints the number.	0.95 for both levels	A standard ED target, and strict enough that the penalty term actually binds. Every table reports both service levels so a different threshold can be applied by eye.
T_{follow} , the delay weight for follow-up patients, needed by the $c\text{-}\mu$ rule. The paper gives no target for them.	60 min — the midpoint of $T_3 = 30$ and $T_4 = 120$	<code>run_03</code> sweeps it from 15 to 120 min. The policy ranking never flips anywhere in the sweep.
The grid-search box for (k_1, k_2) .	$k_1 \in [0,30]$, $k_2 \in [0,60]$	The paper's Table 3 lists winners at $k_1 = 26$ and $k_2 = 40$, so the box must at least contain those.
The cost weights α, β, γ (WP4 only — the paper has no cost model at all).	120 / 0.50 / 50	An ED attending at ~USD 250k/yr loaded over ~2080 h/yr is ~120/h; β values a patient-hour of waiting at 30 units. <code>run_05</code> stage D re-prices everything under four alternative weightings.
'60% require at least one test' versus independent per-machine probabilities that can jointly select none.	Resample so 'at least one' holds exactly	The two readings differ by 1.99% of exam volume. Switchable via <code>SimConfig.exam_require_at_least_one</code> .

10.1 Ambiguities in the paper we had to interpret

Ambiguity	Our reading, and the evidence for it
Is R^t in the policy equations the <i>free</i> doctors or the <i>whole</i> roster?	Free doctors for IFP and SBP (pure priority orders, where the two readings coincide); the whole roster for ALT's $[R/2]$ cap. With event-driven dispatch a doctor usually frees up one at a time, so $[R_{\text{free}}/2] = 0$ would silently turn ALT into 'follow-ups always first' — a different policy. Our reading reproduces the paper's ALT Level III wait (6.82 published, 6.05 ours).
What does 'average waiting time' mean? The equations say per-consultation; the numbers say otherwise.	Per-patient totals (initial wait + follow-up wait). Two facts force it: (i) $0.25 \times 10.48 + 0.75 \times 43.89 = 35.54 \approx 35.25$, the paper's headline; (ii) the paper reports Level III = 44.14 min under IFP and a 100% service level against a 30-min target — both cannot be true of the same quantity. We compute the per-consultation figures too, so nothing is hidden.
How many of each machine?	One. Table 9 lists 'Available Time = 10,080 min' per machine for a one-week run — exactly one machine-week.
How is the truncated exponential parameterised?	$\lambda = 1/\text{mean}$, i.e. the <i>pre</i> -truncation mean is 9/15 min. A truncated exponential on $[5,25]$ cannot have mean 15, and $\lambda = 1/15$ gives $p \approx 0.87$ which matches the paper's ~85% utilisation.
What happens at a shift change?	Nothing is interrupted. 'Once a physician begins service, the service process will not be interrupted.' A shrinking shift blocks new starts until the busy count falls back below the roster.

11. Viva preparation: likely questions

Q. Why discrete-event simulation instead of a queueing formula?

Three reasons, all in the paper and all true of our rebuild. The system is **non-stationary** — arrival rates change hourly and staffing changes by shift — so steady-state formulas do not apply. The service times are **truncated exponentials plus fixed machine times**, which breaks the memorylessness most analytic models need. And the slack-based policy is **history-dependent**: you would have to track every patient's exact waiting time as a state variable, which explodes the state space.

Q. Why write your own event loop instead of using SimPy?

Because the paper's mechanisms are the thing being examined. With a hand-rolled heapq loop, every state transition is one visible line of Python, so the equations in the paper map onto code you can point at. It is also measurably faster: profiling showed dataclass comparison in the heap was 22% of runtime, and switching to plain tuples cut per-replication time from 0.20 s to 0.076 s — which is what makes the 50,415-simulation grid search affordable at all.

Q. Your numbers do not match the paper. Is the model wrong?

No — and the evidence is that everything *except* the absolute level does match: the equipment table to within 0.5 percentage points, ALT's Level III wait to within 11%, both service levels under IFP exactly, and all three sensitivity findings. The absolute waits are higher because the published parameters imply $\rho \approx 0.87$, where mean wait is hypersensitive to utilisation. The underlying dataset is not public, so exact agreement is unattainable; we state the deviation openly rather than tuning to hit it.

Q. Why is the $c \cdot \mu$ rule better than just inventing another heuristic?

Because it comes with a proof. Cox and Smith showed that serving the class with the largest $c_i \cdot \mu_i$ minimises total holding cost in a multi-class queue. IFP, ALT and SBP are plausible rules someone designed; $c \cdot \mu$ is the answer to a stated optimisation problem. The caveat we also state: the proof assumes a single-stage queue, and ours has re-entrant flow and a time-varying server pool, so the guarantee is approximate here.

Q. How can Nelder-Mead work on a random function?

It cannot, as-is — that is why we do not hand it one. We use sample average approximation with common random numbers: every candidate is scored on the *same fixed* 50 simulated weeks, so evaluating a point twice returns the identical number. The optimiser sees a genuine deterministic function, and because the seed noise is shared it cancels when candidates are compared, making the surface far smoother than the pointwise noise would suggest.

Q. Is a 19× speed-up a fair comparison?

Yes, and we went out of our way to make it so. Both methods minimise the same penalised objective, on the same simulated weeks, and the budget is counted in week-long simulations rather than seconds so it does not depend on the machine or the core count. The one trap — that the two methods used different replication counts per point, so their raw objective values are not comparable — is handled by re-scoring both optima on one identical full-length batch before declaring anything.

Q. Why did you get a different optimum from the paper?

Because we optimised a *constrained* problem on a busier system. The paper's (13.1, 2.1) minimises unconstrained mean wait on its own model; on ours it reaches only 89.4% on Level IV. Under the 95% constraint the optimum moves to a much larger k_2 , which is mechanically obvious once you see that $k_2 = 2.1$ rescues a Level IV patient only at minute 117.9 of 120.

Q. Does CRN not bias your results?

No, and we test for it explicitly. CRN changes the *dependence* between two runs, not the marginal distribution of either, so each policy's own mean and standard deviation are statistically unchanged between the two arms — run_06 prints exactly that check. What changes is $\text{Cov}(A,B)$ in $\text{Var}(A-B) = \text{Var}(A) + \text{Var}(B) - 2\text{Cov}(A,B)$, and therefore only the variance of the *difference*.

Q. Where do the cost weights come from? Are they not arbitrary?

They are assumptions, stated as assumptions, and stress-tested. $\alpha = 120/\text{hour}$ is an ED attending on ~USD 250k/yr loaded over ~2080 h/yr. β and γ are choices. That is why stage D of run_05 re-prices every roster under four alternative weightings — and the winning roster does change, which is itself the finding: the right staffing level depends on how the hospital prices waiting against salary.

Q. How do I know the simulator is actually correct?

33 automated checks, each pinning down a property some published number depends on. They verify the samplers against closed forms, the day/hour indexing, thinning convergence, CRN seed behaviour, and engine invariants — no patient served before they arrive, no doctor idle while someone waits, the exam chain matching the paper's equation 4, non-preemptive shift changes. Plus the optimiser against Rosenbrock and against SciPy, and a direct check that CRN reduces variance.

Q. Which policy should the hospital actually use?

SBP* — the Slack-Based Policy with our re-tuned thresholds — if they must meet a 95% service level, which in practice they must. It is 16% faster than the IFP baseline and it is the only tuned policy that clears the constraint on both triage levels. If the constraint were dropped, WSEPT would be the answer. And WP4 says the bigger win is not the schedule at all: one extra doctor on each day shift saves 20.7% of weekly cost.

12. Glossary and references

12.1 Symbols and terms

Symbol / term	Means
$\lambda_{d,h}$	Arrival rate on day d , hour h (patients/hour). The paper's Table 2.
λ^*	The largest λ anywhere in the week (23.05/h), used by the thinning algorithm.
ρ (rho)	Offered load = work arriving / work the doctors can do. 0.87 here; at 1.0 the queue grows without bound.
T_ℓ	Triage target for level ℓ : 30 min for Level III, 120 min for Level IV.
k_1, k_2	Slack thresholds. A patient becomes urgent after waiting $T_\ell - k_\ell$ minutes.
SL_ℓ	Service level: the share of level- ℓ patients seen within T_ℓ .
μ_ℓ	Service rate = 1 / mean service time for class ℓ .
c_ℓ	Delay cost per minute for class ℓ . We use $1/T_\ell$.
Holding cost	Mean of $w_i / T_{\text{level}(i)}$ — waiting measured in units of each patient's own tolerance.
CRN	Common random numbers: giving every policy the identical simulated week.
SAA	Sample average approximation: replacing a random objective by its average over a fixed set of scenarios.
Replication	One complete simulated week (10,080 minutes, ~2,280 patients).
IFP / ALT / SBP	The paper's three policies. SBP* is our re-tuned version of SBP.
WSEPT / Gc· μ	Our two added policies: the c· μ rule and its generalisation.

12.2 References

- [1] Lv, W.; Liu, R.; Yan, F.; Wang, Y. *Discrete Event Simulation-Based Analysis and Optimization of Emergency Patient Scheduling Strategies*. Healthcare **2026**, 14, 99. — the base paper.
- [2] Cox, D. R.; Smith, W. L. *Queues*. Methuen, 1961. — the c· μ rule.
- [3] Buyukkoc, C.; Varaiya, P.; Walrand, J. *The c μ rule revisited*. Adv. Appl. Prob. **1985**, 17, 237–238.
- [4] Van Mieghem, J. A. *Dynamic scheduling with convex delay costs: the generalized c μ rule*. Ann. Appl. Prob. **1995**, 5, 809–833.
- [5] Mandelbaum, A.; Stolyar, A. L. *Scheduling flexible servers with convex delay costs*. Oper. Res. **2004**, 52, 836–855.
- [6] Nelder, J. A.; Mead, R. *A simplex method for function minimization*. Comput. J. **1965**, 7, 308–313.
- [7] Lewis, P. A. W.; Shedler, G. S. *Simulation of nonhomogeneous Poisson processes by thinning*. Naval Res. Logist. Q. **1979**, 26, 403–413.
- [8] Law, A. M. *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill, 2015. — common random numbers.

Where to go next

edsim/README.md — the same material in written form, with more detail on the code.
edsim/docs/PAPER_MAPPING.md — every equation, table and figure of the paper mapped to the exact function that implements it. **edsim/docs/METHODS.md** — the numerical methods with full formulas.
edsim/results/RESULTS_SUMMARY.md and **PROVENANCE.md** — the generated numbers and the record of exactly which command produced each one.